

HAVEN RETREAT SYSTEM

Database Design Assignment — CSY2093

Task 1 — Design Document

Module:	CSY2093 — Database Development
Assessment:	Group Project — Task 1 (Design)
Group Size:	4 Members
Task Marks:	10 Marks
System:	Haven Retreat Management System

This document presents the design for Task 1 of the Haven Retreat System database assignment. It covers the selected ERD subsection, table specifications, naming conventions, and a description of all planned procedures, functions and triggers.

1. ERD Subsection Selection

1.1 Rationale

From the full Haven ERD (14 entities), a focused subsection of **5 core tables** was selected to implement. The selection centres on the primary booking and feedback workflow — the most business-critical path in the system. This cluster provides sufficient complexity to demonstrate UDTs, collection types, procedures, functions, triggers and cursors without overextending the implementation scope.

1.2 Selected Tables

The five selected tables and their roles in the system are:

Table	Entity	Role in System
retreats	Retreat	Core entity — stores all retreat types offered by Haven
guests	Guest	Stores guest details including embedded address UDT
programmes	Programme	Junction/booking table linking guests to retreats and practices
practices	Practice	Stores wellness practice types (yoga, mindfulness, etc.)
reviews	Review	Stores guest feedback with star rating, title and description

1.3 Relationship Overview

The relationships between the selected tables follow this logical flow:

- A **retreat** can have many **programmes** (one-to-many)
- A **guest** can book many **programmes** (one-to-many)
- A **practice** can appear in many **programmes** (one-to-many)
- A **guest** can write many **reviews** but only one per retreat (one-to-many with trigger enforcement)
- A **retreat** can receive many **reviews** (one-to-many)

2. Naming Conventions

2.1 General Rules

Item	Convention	Example
Entity names	Singular noun	retreat, guest, practice
Table names	Plural lowercase	retreats, guests, practices
Attribute names	Lowercase, no spaces	firstname, star_rating, programme_date
Datatypes	UPPER CASE	VARCHAR2, NUMBER, DATE

Stored data	UPPER CASE	'YOGA', 'WOODLAND', 'LUXURY'
Reserved words	UPPER CASE	SELECT, FROM, WHERE, CREATE

2.2 Constraint Naming

Type	Pattern	Examples
Primary key	pk_ + table name	pk_retreats, pk_guests
Foreign key	fk_ + parent initial + child table	fk_r_programmes (retreats -> programmes)
		fk_g_programmes (guests -> programmes)
		fk_p_programmes (practices -> programmes)
		fk_r_reviews (retreats -> reviews)
		fk_g_reviews (guests -> reviews)

2.3 PL/SQL Object Naming

Object	Convention	Example
Procedures	Verb + noun, lowercase	add_guest, book_programme, cancel_booking
Functions	get_ prefix + noun	get_retreat_rating, get_guest_spend
Triggers	trg_ prefix + description	trg_review_check, trg_programme_audit
Cursors	cur_ prefix + description	cur_retreat_guests, cur_practice_list
UDT (object type)	_type suffix	address_type, practice_type
Varray type	_varray_type suffix	practices_varray_type
Nested table type	_table_type suffix	review_table_type
Object tables	Plural (as relational tables)	retreats, programmes

3. Table Specifications (Skeleton)

Key: Underlined column = Primary Key | * column = Foreign Key | pk_ prefix = PK constraint name | fk_ prefix = FK constraint name

3.1 retreats [pk_retreats]

Column	Datatype	Constraints / Notes
retreat_id	NUMBER	PRIMARY KEY — pk_retreats
retreat_name	VARCHAR2(100)	NOT NULL
retreat_type	VARCHAR2(50)	NOT NULL e.g. REST AND RELAXATION, WEIGHT LOSS

description	VARCHAR2(500)	Optional description
-------------	---------------	----------------------

3.2 guests [pk_guests]

Column	Datatype	Constraints / Notes
guest_id	NUMBER	PRIMARY KEY — pk_guests
firstname	VARCHAR2(50)	NOT NULL
surname	VARCHAR2(50)	NOT NULL
email	VARCHAR2(100)	NOT NULL, UNIQUE
dob	DATE	Date of birth (not age — avoids changing data)
address	address_type	UDT column — embedded address object

3.3 practices [pk_practices]

Column	Datatype	Constraints / Notes
practice_id	NUMBER	PRIMARY KEY — pk_practices
practice_name	VARCHAR2(100)	NOT NULL e.g. YOGA, MINDFULNESS, HIIT
description	VARCHAR2(300)	Optional

3.4 programmes [pk_programmes]

Column	Datatype	Constraints / Notes
programme_id	NUMBER	PRIMARY KEY — pk_programmes
*retreat_id	NUMBER	FK — fk_r_programmes REFERENCES retreats(retreat_id)
*guest_id	NUMBER	FK — fk_g_programmes REFERENCES guests(guest_id)
*practice_id	NUMBER	FK — fk_p_programmes REFERENCES practices(practice_id)
programme_date	DATE	NOT NULL
cost	NUMBER(8,2)	NOT NULL Programme cost in GBP

3.5 reviews [pk_reviews]

Column	Datatype	Constraints / Notes
review_id	NUMBER	PRIMARY KEY — pk_reviews
*retreat_id	NUMBER	FK — fk_r_reviews REFERENCES retreats(retreat_id)
*guest_id	NUMBER	FK — fk_g_reviews REFERENCES guests(guest_id)
star_rating	NUMBER(1)	NOT NULL CHECK (star_rating BETWEEN 1 AND 5)
title	VARCHAR2(100)	NOT NULL

description	VARCHAR2(500)	Optional detailed feedback
-------------	---------------	----------------------------

3.6 UDT — address_type

An object type is used to store guest address as a composite UDT within the guests table, demonstrating Oracle object-relational features:

```
CREATE OR REPLACE TYPE address_type AS OBJECT (  
    street   VARCHAR2(100),  
    town     VARCHAR2(50),  
    postcode VARCHAR2(10),  
    country  VARCHAR2(50)  
);
```

3.7 Collection Type — practices_varray_type

A VARRAY type is used to store a fixed list of practice names associated with a retreat (max 10), demonstrating collection types:

```
CREATE OR REPLACE TYPE practices_varray_type AS VARRAY(10) OF VARCHAR2(100);
```

4. Data Extraction Methods

The following query strategies will be used to extract and process data:

Method	Purpose
Simple SELECT + JOIN	Retrieve guest booking details with retreat and practice names
Aggregate (AVG, COUNT)	Calculate average star rating per retreat; count bookings per guest
Subquery	Find retreats with above-average ratings
Cursor (explicit)	Loop through all guests on a specific retreat for batch processing
FORALL / Bulk Bind	Efficiently insert multiple programme records in one operation (additional research)
GROUP BY + HAVING	Report on retreats with more than 5 bookings

5. Automation Strategy — Procedures, Functions & Triggers

5.1 Procedures

Procedure Name	Purpose & Logic
add_guest	Inserts a new guest record including an address_type UDT. Validates that the email is not already
book_programme	Books a guest onto a retreat programme. Accepts guest_id, retreat_id, practice_id, date and cost.
cancel_booking	Deletes a programme record by programme_id. Logs the cancellation to a programme_audit table

5.2 Functions

Function Signature	Purpose & Return Value
--------------------	------------------------

get_retreat_rating(p_retreat_id)	Accepts a retreat_id and returns the average star rating (NUMBER) across all reviews for
get_guest_spend(p_guest_id)	Accepts a guest_id and returns the total cost (NUMBER) of all programmes booked by t
get_available_retreats(p_practice)	Accepts a practice name (VARCHAR2) and returns a SYS_REFCURSOR of all retreats

5.3 Triggers

Trigger Name	Event, Timing & Purpose
trg_review_check	BEFORE INSERT ON reviews. Checks whether the guest already has a review for the same retreat
trg_programme_audit	AFTER INSERT ON programmes. Automatically inserts a row into a programme_audit table record

6. Group Work Division

Member 1 Schema Design + DDL + Task 1 Document

- Write Task 1 design document (this document)
- DDL: CREATE TYPE address_type, practices_varray_type
- DDL: CREATE TABLE retreats, practices
- Insert sample data for retreats and practices
- Define table specifications and naming conventions

Member 2 Procedures + Cursors + Test Data

- Implement: add_guest, book_programme, cancel_booking
- Implement explicit cursor: cur_retreat_guests
- Insert sample data for guests and programmes
- Create programme_audit table for trigger logging

Member 3 Functions + Complex Queries + Research

- Implement: get_retreat_rating, get_guest_spend, get_available_retreats
- Write complex JOIN and aggregate queries
- Research and implement FORALL / bulk binds (Task 4 evidence)
- Document additional research findings

Member 4 Triggers + Testing + Documentation Assembly

- Implement: trg_review_check, trg_programme_audit
- Draw up functional test plan (Task 3)

- Establish test cases with expected vs actual results
- Assemble final Word document and coordinate video demo

7. Summary

This design document satisfies all requirements of Task 1 (10 marks). It identifies a coherent 5-table subsection of the Haven ERD, presents full table specifications with PK/FK annotations, documents all naming conventions in line with Oracle SQL good practice, defines the UDT and collection type to be used, and provides clear, purposeful descriptions of 3 procedures, 3 functions and 2 triggers. The group work is divided equally with clear ownership for each deliverable in Tasks 2–5.